

1 Selection Sort

Algorithm 1: Selection Sort

Data: Sequence of numbers (unsorted) with length elements

Result: Sequence of numbers (sorted)

currentPosition $\leftarrow$ length-1;

while currentPosition > 0 **do**

 biggestElementPosition $\leftarrow$ maxIndex(sequence[0..currentPosition]);

 swap(sequence[currentPosition], sequence[biggestElementPosition]);

 currentPosition $\leftarrow$ currentPosition - 1;

end

2 Insertion Sort

Algorithm 2: Insertion Sort

Data: Sequence of numbers (unsorted) with length elements

Result: Sequence of numbers (sorted)

for iterationNumber $\leftarrow$ 1 **to** length-1 **do**

 firstUnsortedElement $\leftarrow$ sequence[iterationNumber];

 insertionPosition $\leftarrow$ i;

 /* Find correct insertion position by iterating to the start of the sequence */

while insertionPosition > 0 **do**

 /* Push element to the start as long as it is smaller than the current element in the sequence */

if sequence[insertionPosition-1] > firstUnsortedElement **then**

 sequence[insertionPosition] $\leftarrow$ sequence[insertionPosition-1];

 insertionPosition $\leftarrow$ insertionPosition-1;

else

 break;

end

 sequence[insertionPosition] $\leftarrow$ firstUnsortedElement;

end

end

3 Quicksort

Algorithm 3: Quicksort

Algorithm Quicksort(sequence, lowerBoundPosition, upperBoundPosition)

```
    if upperBoundPosition > lowerBoundPosition then
        pivotPosition ← determine pivot position;
        newPivotPosition ← Partition(sequence, upperBoundPosition,
            lowerBoundPosition, pivotPosition);
        Quicksort(sequence, lowerBoundPosition, newPivotPosition-1);
        Quicksort(sequence, newPivotPosition+1, upperBoundPosition);
    end
```

Procedure Partition(sequence, lowerBoundPosition, upperBoundPosition, pivotPosition)

```
    newPivotPosition ← lowerBoundPosition;
    pivotValue ← sequence[pivotPosition];
    swap(sequence[pivotPosition], sequence[upperBound]);
    for i ← lowerBound to upperBoundPosition-1 do
        if sequence[i] < pivotValue then
            swap(sequence[newPivotPosition], sequence[i]);
            newPivotPosition ← newPivotPosition+1;
        end
    end
    swap(sequence[upperBoundPosition], sequence[newPivotPosition]);
    return;
```
